

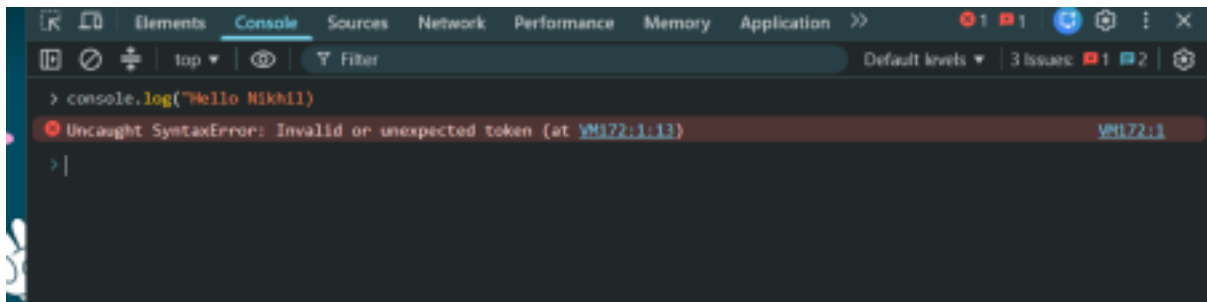
Assignment of JavaScript

Part-1 (Syntax Warm-Up)

Task 1.1: Deliberately break the syntax — remove the closing quote from a string

and press Enter. Read the error message carefully, then fix it.

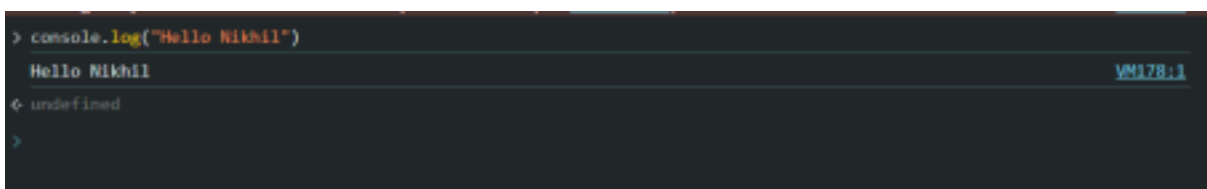
AWNSER



This is the error message I get after breaking the syntax.

FIXING IT

Put the string in inside quotes and use both opening and closing quotes for solving the syntax error issue.



Task 1.2 (pair discussion): Type `console.log("test");`. Now imagine you had instead written `console.log("Test");` somewhere expecting it to match a variable called `test`.

Why would that fail? Discuss case sensitivity with a partner and write one sentence explaining it.

AWNSER

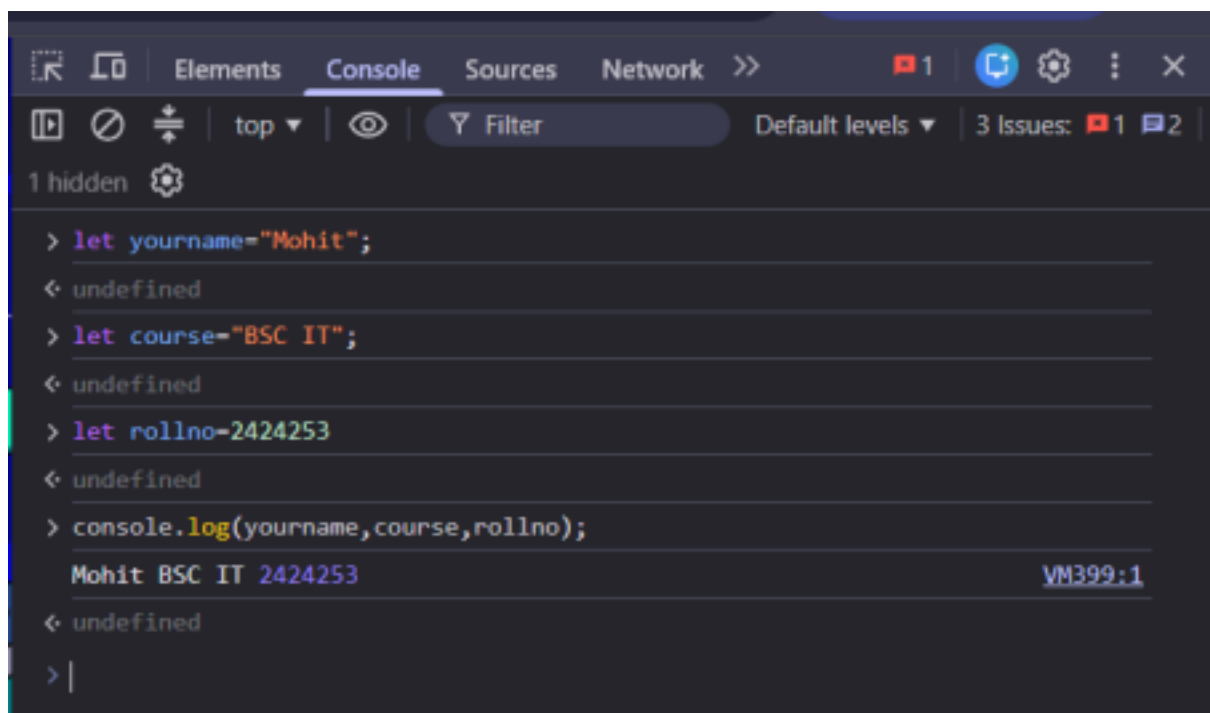
JavaScript is case-sensitive, so `test` and `Test` are treated as different names, meaning `Test` will not

match a variable called test.

Part 2 — Variables (20 min)

Task 2.1: Declare three let variables: your name, your roll number, and your course (BCA or BIT). Print all three in a single console.log.

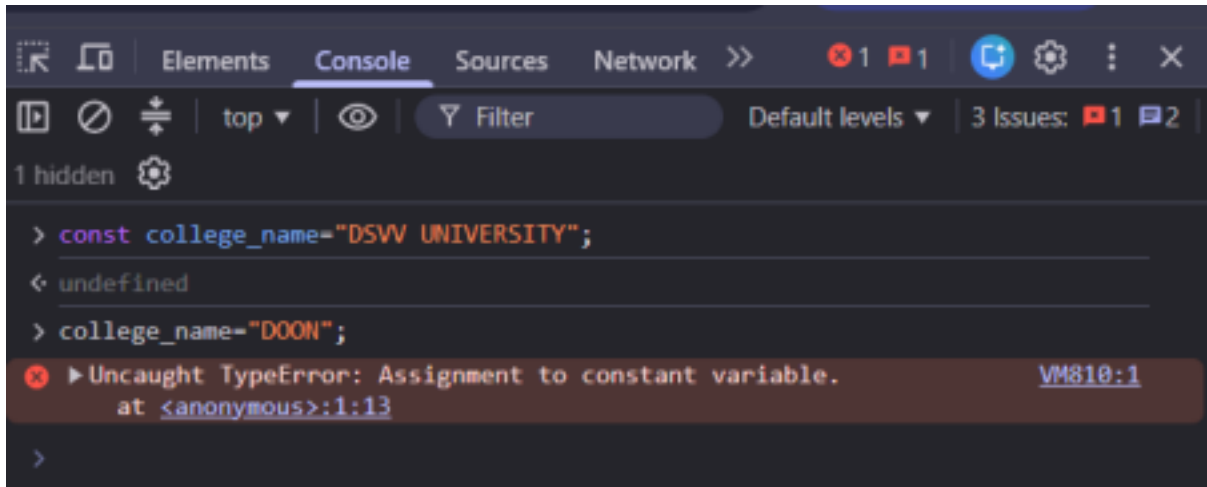
ANSWER



```
> let yourname="Mohit";  
← undefined  
> let course="BSC IT";  
← undefined  
> let rollno=2424253  
← undefined  
> console.log(yourname,course,rollno);  
Mohit BSC IT 2424253  
← undefined  
> |
```

Task 2.2: Try reassigning collegeName to a different value. Write down the exact error message you get.

ANSWER



This is the error message I got = VM810:1 Uncaught TypeError: Assignment to constant variable. at <anonymous>:1:13

Task 2.3 — Naming Practice: For each of the following, write "valid" or "invalid"

and explain why:

let 1stYear; let student_name; let let; let \$marks; let roll no;

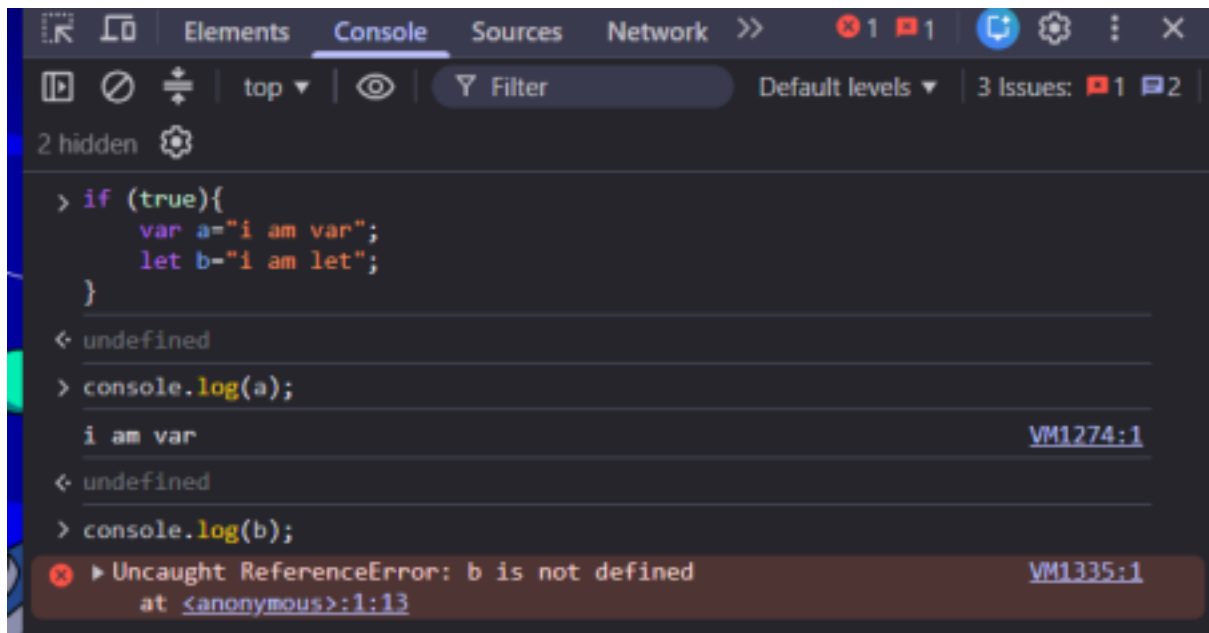
ANSWER

1. let 1stYear; Invalid Cannot use no. in starting of variable declaration.
2. let student_name; Valid (_) is allowed
3. let let; Invalid Let is keyword not allowed, 4. let \$marks; Valid \$ sign is allowed.
5. let roll no; Invalid spaces are not allowed

Task 2.4 — Scope Challenge: Run the code below. One line works, one throws an error. In your lab record, explain why, in your own words.

```
if (true) {  
  var a = " I am var ";  
  let b = " I am let";  
}  
console.log(a);  
console.log(b);
```

ANSWER



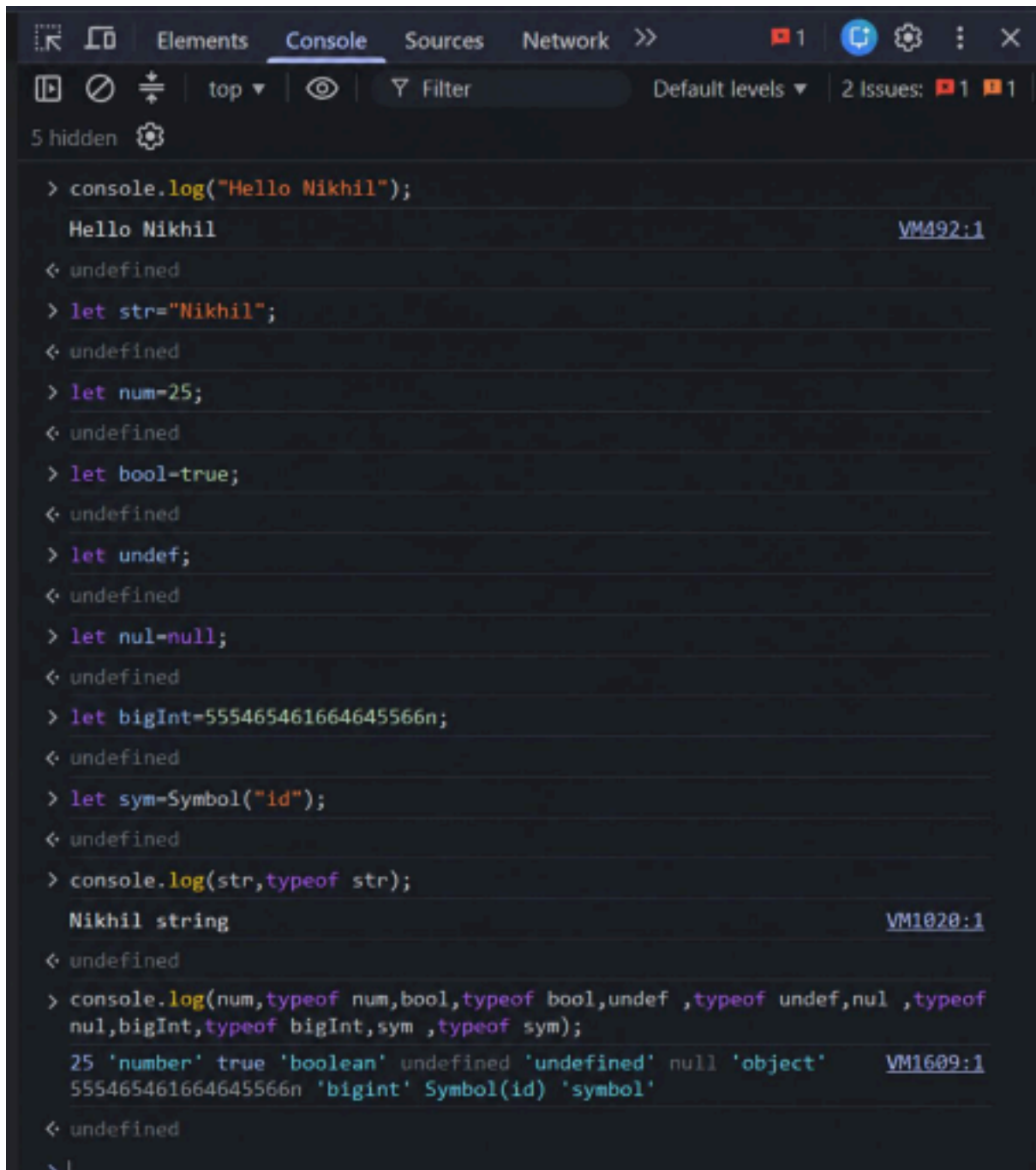
```
> if (true){  
    var a="i am var";  
    let b="i am let";  
}  
  
< undefined  
> console.log(a);  
i am var VM1274:1  
  
< undefined  
> console.log(b);  
  
✖ ▶ Uncaught ReferenceError: b is not defined VM1335:1  
   at <anonymous>:1:13
```

var is function-scoped, so a can be accessed outside the if block. let is block-scoped, so b only exists inside the if block and causes an error when accessed outside it.

Part 3 — Data Types

Task 3.1: Create one variable for each of the 7 primitive types (String, Number, Boolean, Undefined, Null, BigInt, Symbol) and print each one together with its type of result.

ANSWER

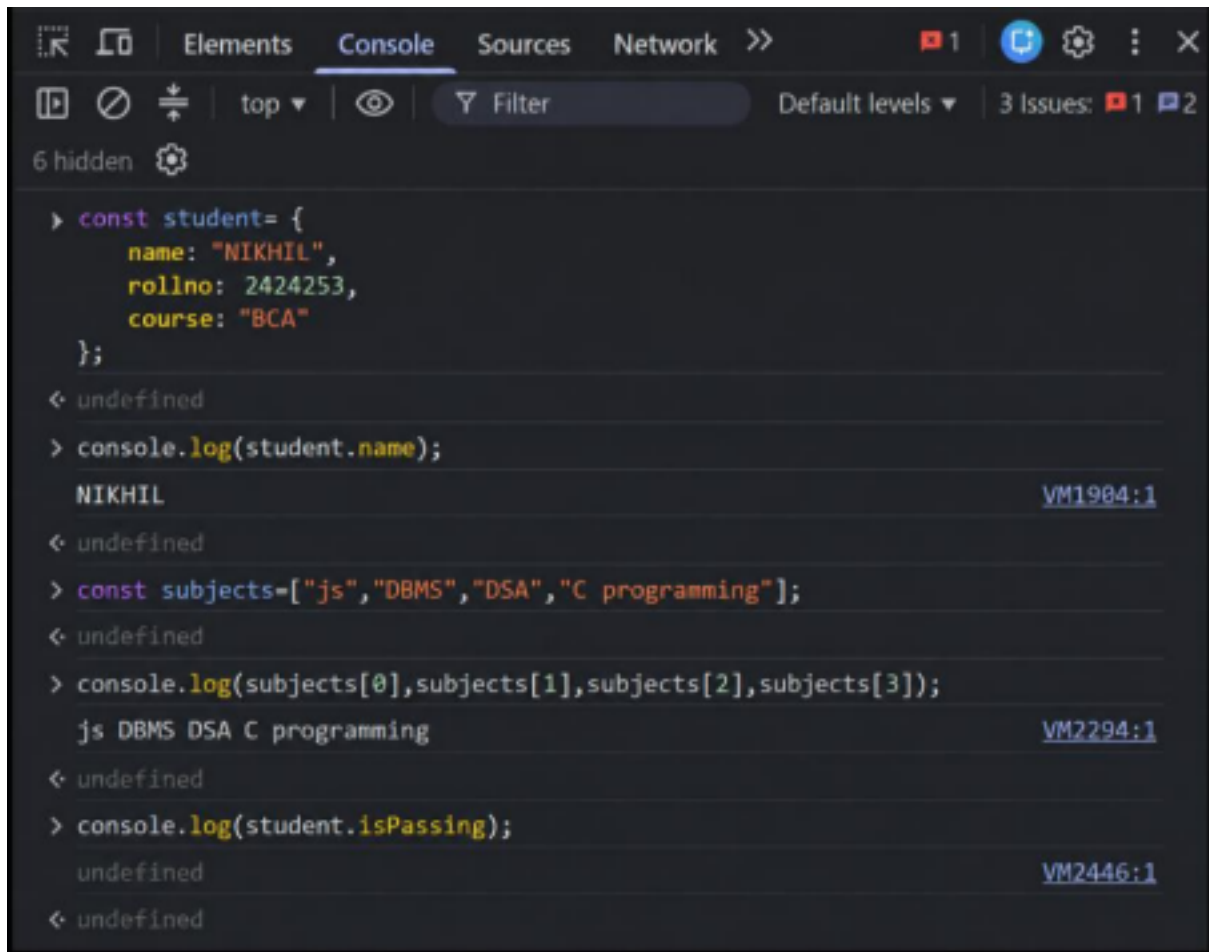
A screenshot of a web browser's developer console. The console is open, showing a series of JavaScript commands and their outputs. The commands include logging a string, declaring variables with 'let', and logging the types of these variables. The outputs show the string 'Hello Nikhil', 'undefined' for most variable declarations, and a detailed list of variable types for the final log statement. The console interface includes tabs for Elements, Console, Sources, and Network, along with a filter and issue count at the top.

```
> console.log("Hello Nikhil");
Hello Nikhil VM492:1
< undefined
> let str="Nikhil";
< undefined
> let num=25;
< undefined
> let bool=true;
< undefined
> let undef;
< undefined
> let nul=null;
< undefined
> let bigInt=555465461664645566n;
< undefined
> let sym=Symbol("id");
< undefined
> console.log(str,typeof str);
Nikhil string VM1020:1
< undefined
> console.log(num,typeof num,bool,typeof bool,undef ,typeof undef,nul ,typeof
nul,bigInt,typeof bigInt,sym ,typeof sym);
25 'number' true 'boolean' undefined 'undefined' null 'object' VM1609:1
555465461664645566n 'bigint' Symbol(id) 'symbol'
< undefined
> |
```

Task 3.2 — Object & Array: Run the code below, then modify it: add a 4th subject to the array and print it. Add an isPassing property to the student object and print it.

```
const student = {
  name: "Rohit",
  rollNo: 21,
  course: "BCA",
};
console.log(student.name);
const subjects = ["JavaScript", "DBMS", "Networking"];
console.log(subjects[0], subjects[2]);
```

ANSWER

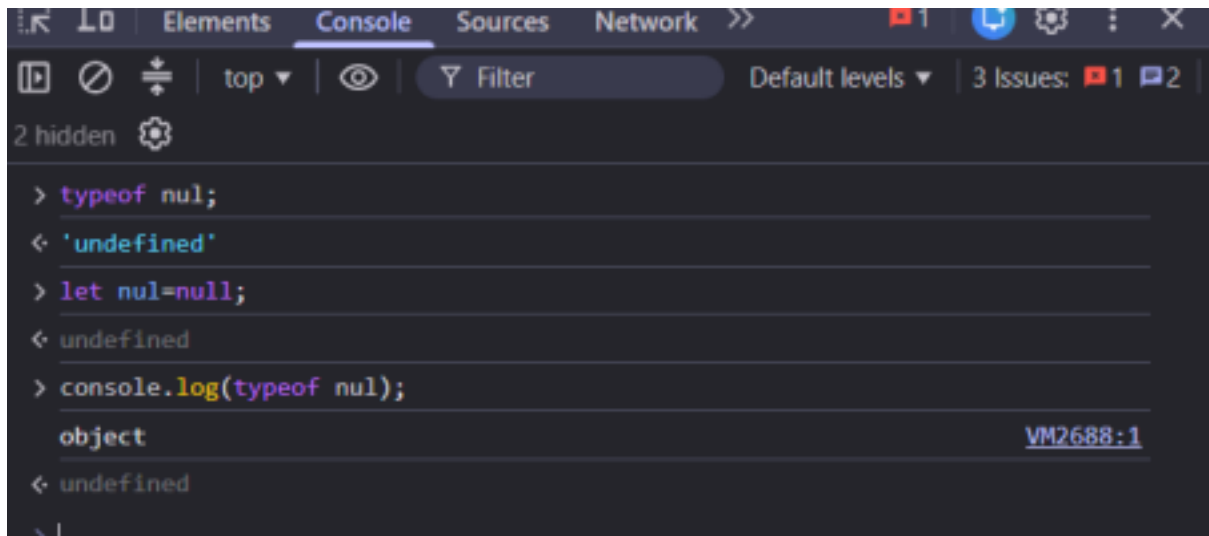


```
> const student= {  
  name: "NIKHIL",  
  rollno: 2424253,  
  course: "BCA"  
};  
← undefined  
> console.log(student.name);  
NIKHIL VM1984:1  
← undefined  
> const subjects=["js","DBMS","DSA","C programming"];  
← undefined  
> console.log(subjects[0],subjects[1],subjects[2],subjects[3]);  
js DBMS DSA C programming VM2294:1  
← undefined  
> console.log(student.isPassing);  
undefined VM2446:1  
← undefined
```

Task 3.3 — The typeof null Trick: Predict typeof null before running it. Were you right? Write one sentence explaining why it behaves this way.

ANSWER

typeof null will result "object" because of the historical bug which was not solved because of compatibility issue.



```
> typeof nul;
< 'undefined'

> let nul=null;
< undefined

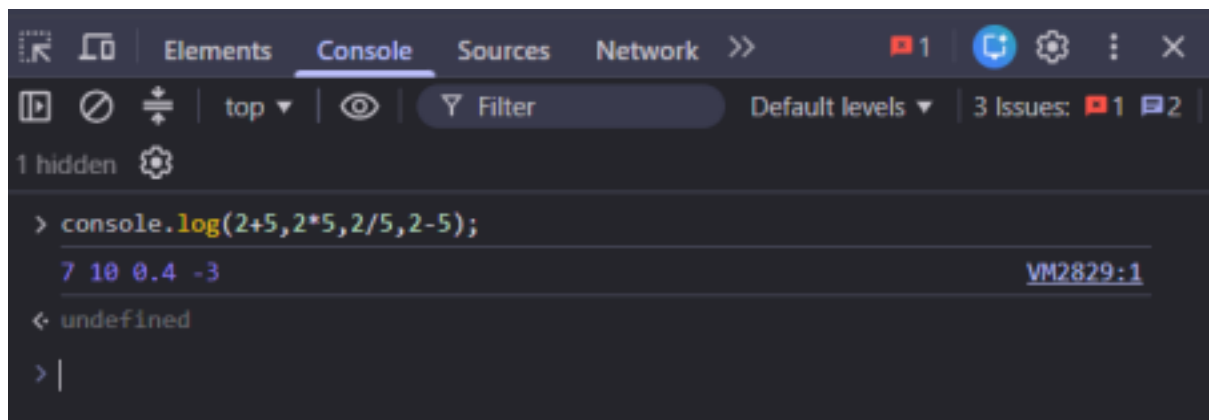
> console.log(typeof nul);
object VM2688:1

< undefined
```

Part 4 — Operators & Expressions

Task 4.1 — Arithmetic: Pick two numbers and print their sum, difference, product, quotient, and remainder.

ANSWER



```
> console.log(2+5,2*5,2/5,2-5);
7 10 0.4 -3 VM2829:1

< undefined

> |
```

Task 4.2 — == vs === Investigation: Run all six of the following and record

true/false for each. Then explain in your own words why the == and === results differ for each pair.

5 == "5"

5 === "5"

0 == false

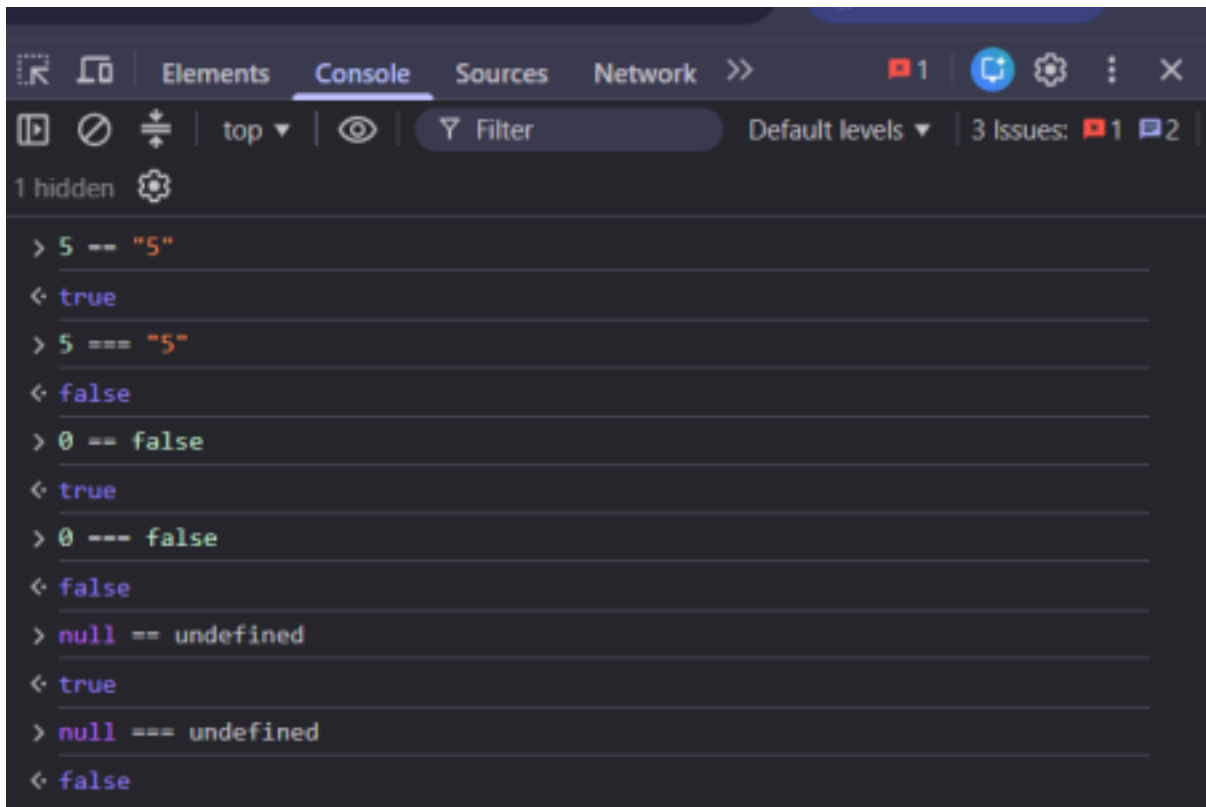
0 === false

null == undefined

null === undefined

ANSWER

1. `5 == "5"` true this convert string "5" to a number before comparing.
2. `5 === "5"` false `===` checks both value **and type**; number and string are different types.
3. `0 == false` true `==` convert false to 0 before comparing. `4.0 === false` false `===` checks the types ; 0 is an no. and false is Boolean
5. `null == undefined` true With `==`, null and undefined are considered equal.
6. `null === undefined` false `===` requires the same type; null and undefined have different types.



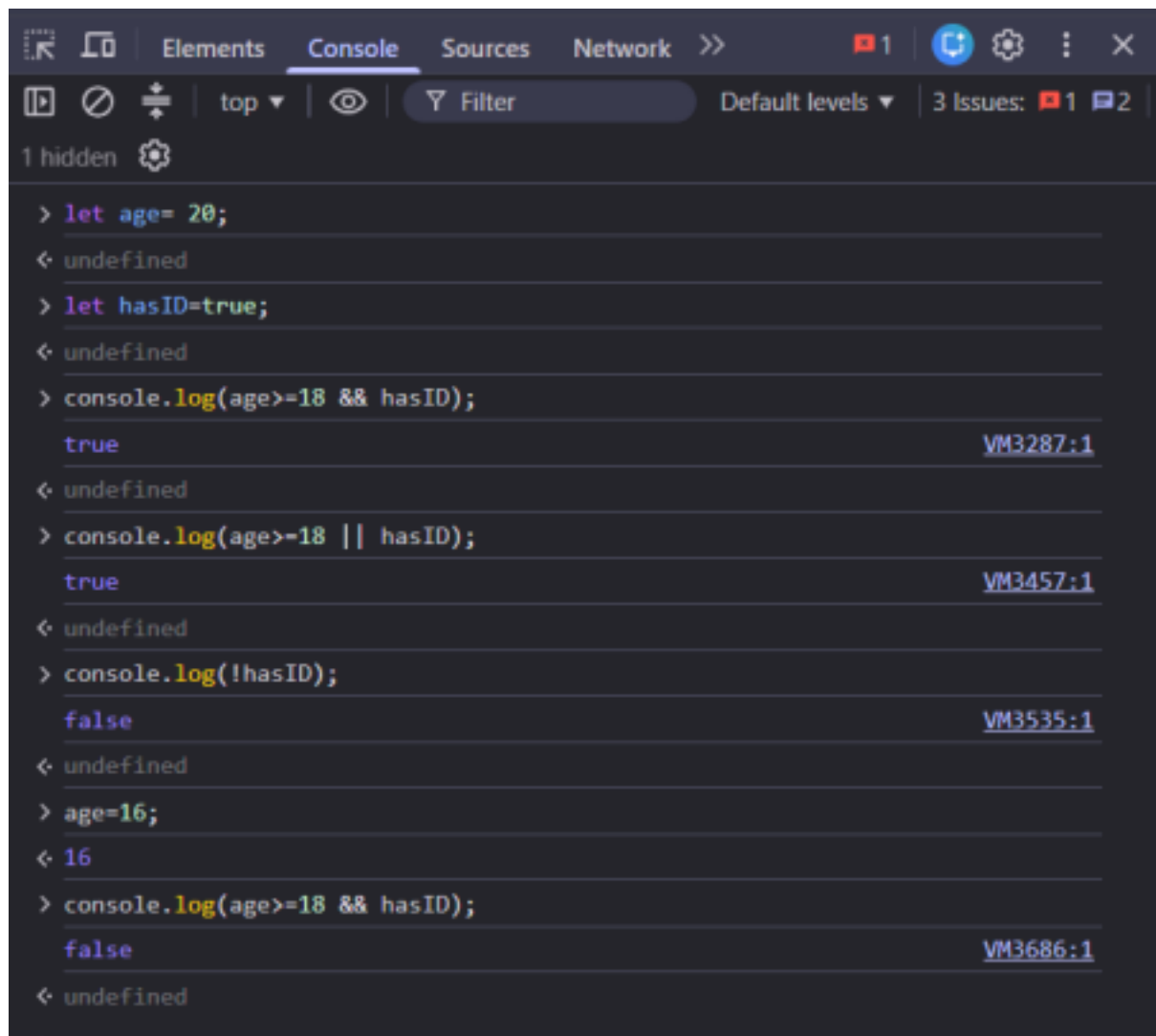
The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following JavaScript code and its results:

```
> 5 == "5"
< true
> 5 === "5"
< false
> 0 == false
< true
> 0 === false
< false
> null == undefined
< true
> null === undefined
< false
```

Task 4.3 — Logical Operators: Run the code below, then change the values of age and hasID until the first line prints false. Record the values you used.

```
let age = 20;
let hasID = true;
console.log(age >= 18 && hasID);
console.log(age >= 18 || hasID);
console.log(!hasID);
```

ANSWER

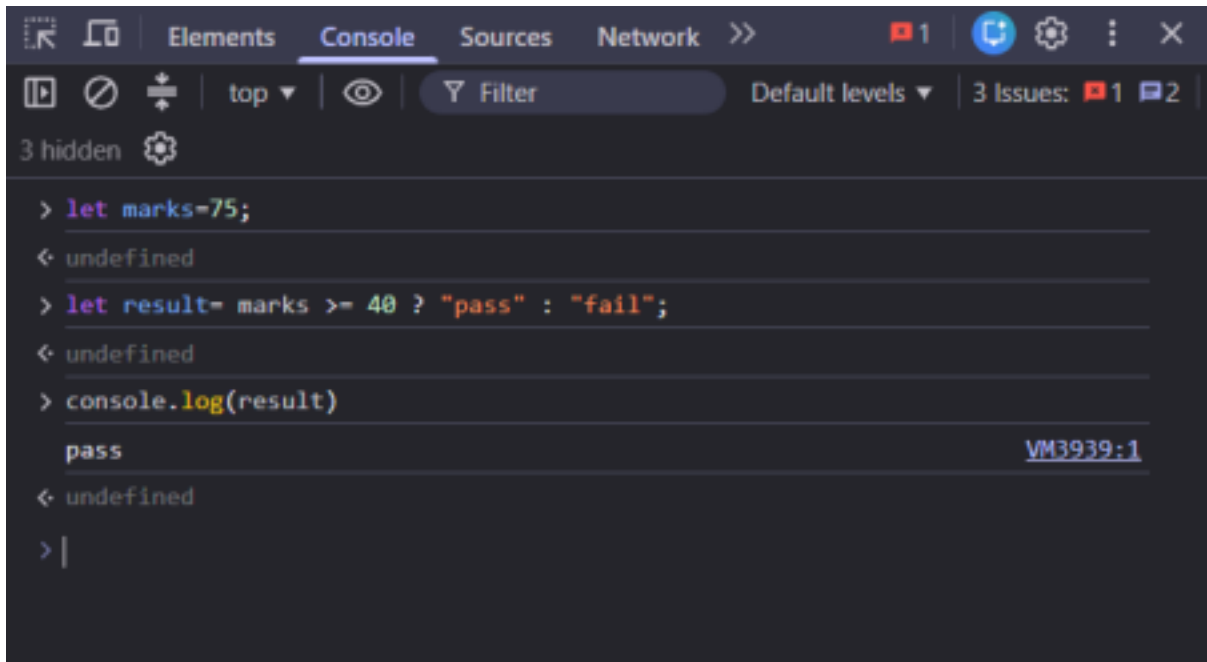


The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following sequence of commands and results:

```
> let age= 20;
< undefined
> let hasID=true;
< undefined
> console.log(age>=18 && hasID);
true VM3287:1
< undefined
> console.log(age>=18 || hasID);
true VM3457:1
< undefined
> console.log(!hasID);
false VM3535:1
< undefined
> age=16;
16
> console.log(age>=18 && hasID);
false VM3686:1
< undefined
```

Task 4.4 — Ternary Operator: Write a ternary expression that stores "Pass" in a variable if a marks variable is ≥ 40 , otherwise "Fail". Test it with two different values of marks.

ANSWER



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following JavaScript code and its output:

```
> let marks=75;
< undefined
> let result= marks >= 40 ? "pass" : "fail";
< undefined
> console.log(result)
pass VM3939:1
< undefined
> |
```

Part 6 — Debugging Practice (10 min)

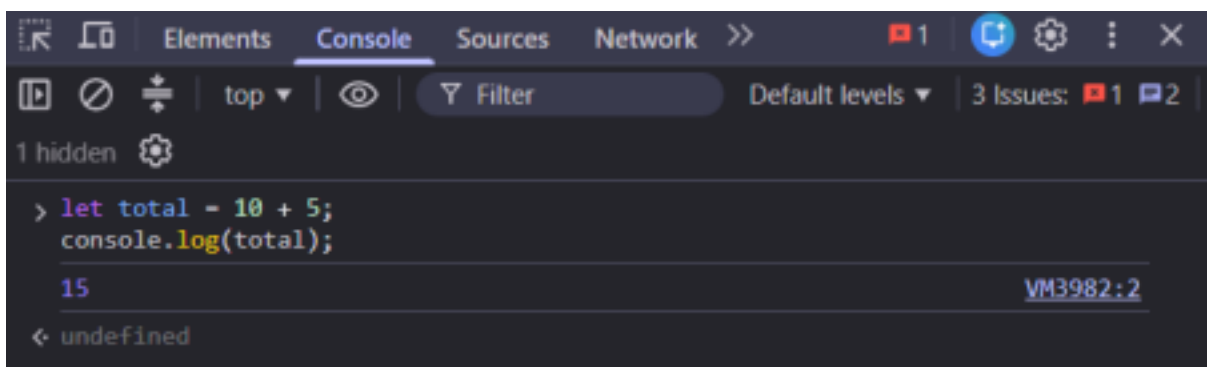
Snippet 1

let total = 10 +

Page 5 of 5

console.log(total);

fixing snippet



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following JavaScript code and its output:

```
> let total = 10 + 5;
  console.log(total);
15 VM3982:2
< undefined
```

Snippet 2

const rate = 5;

rate = 10;

fixing snippet

```
> let rate = 5;
   rate = 10;
< 10
> |
```

Error: TypeError: Assignment to constant variable.

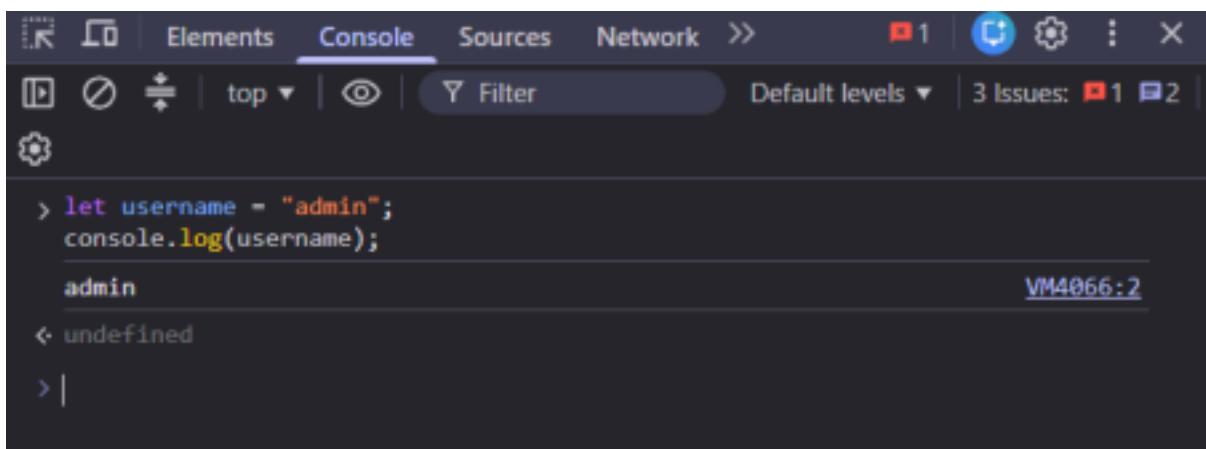
Reason: A variable declared with const cannot be reassigned.

Snippet 3

```
console.log(username);
```

```
let username = "admin";
```

fixing snippet



Error: ReferenceError: Cannot access 'username' before initialization **Reason:** A let variable cannot be accessed before its declaration.

Snippet 4

```
let 2ndAttempt = "retry";
```

fixing this:



Error: SyntaxError: Invalid or unexpected token

Reason: A JavaScript variable name cannot start with a number.

Mini Project: Simple Grade Calculator



